

JobNest

Recruiter-first job platform for private, scalable candidate sourcing

Purpose. JobNest gives a private recruiter a focused web workspace to publish openings, receive structured candidate applications, review professional profiles, and manage hiring progress without maintaining a large social network. It is designed for a low-cost Firebase free-tier launch while keeping a clear path to production scaling.

Product overview

JobNest connects two user groups:

- **Candidates** create a profile, provide contact and professional information, add resume and introduction-video links, browse jobs, and apply.
- **Recruiters** request access, wait for approval, publish job openings, and review applications related to their jobs.
- **Owner administrator** approves recruiters and performs privileged operations through a Firebase custom claim.

Core features

- Email/password authentication plus Google and GitHub sign-in.
- Candidate and recruiter roles with recruiter approval workflow.
- Public job browsing with search, location, role, and status filters.
- Recruiter job creation, management, and application review.
- Application stages: *new*, *reviewing*, *shortlisted*, and *rejected*.
- Candidate dashboard with profile and application metrics.
- In-app reminders and realtime dashboard updates.
- Responsive Next.js interface with reusable shared components.

Technology stack

Next.js 16, React 19, TypeScript, Tailwind CSS, Firebase Authentication, Cloud Firestore, realtime listeners, and composite indexes.

Working procedure

1. A user registers or signs in through Firebase Authentication.
2. The authenticated identity is synchronized with the Firestore profile.
3. A candidate completes a profile and supplies optional HTTPS resume/video links.
4. A recruiter registers and remains pending until the owner approves the account.
5. An approved recruiter creates an opening. Open jobs are publicly readable.
6. A candidate opens a job, submits a cover note and links, and creates an application.
7. The recruiter reviews related applications and updates their stage.
8. The candidate tracks progress and reminders from the dashboard.

Link-based candidate attachments

Resumes and introduction videos use validated **HTTP/HTTPS links** from Google Drive, Dropbox, YouTube, or another trusted host. Only the link is stored, avoiding Firebase Storage billing in the current free-tier design. Candidates control external sharing permissions.

Security and access control

- Firestore rules require authentication for private records and scope access by user relationship.
- Only approved recruiters can create and manage their own jobs.
- Applications are visible to the related candidate, recruiter, or administrator.
- Reminders are restricted to their recipient; owner actions require `admin == true`.
- No passwords, sessions, service-account keys, or local JSON data are stored in the app.

Operations and delivery

Deployment, scaling, validation, and responsible use

Deployment procedure

1. Create a Firebase project and web app.
2. Copy the public Firebase web configuration into a local `.env.local` file.
3. Enable Email/Password, Google, and GitHub providers.
4. Create Firestore and deploy the repository's rules and indexes.
5. Add the production domain to Firebase Authentication authorized domains.
6. Deploy the Next.js application with the production Firebase environment variables.
7. Set the owner's administrator claim through a trusted server-side administrative process.

Validation checklist

- Run `npm run lint`.
- Run `npx tsc --noEmit`.
- Run `npm run build`.
- Run `npm audit --omit=dev`.
- Test registration and sign-in for all enabled providers.
- Test recruiter approval, job creation, application submission, status changes, reminders, and realtime updates using two browser sessions.

Scaling approach

Firestore queries and listeners are scoped and bounded rather than reading entire collections. The free-tier launch should use quota alerts, App Check, authorized domains, abuse monitoring, backups, and a production rollback build. As usage grows, review read/write costs first, then increase Firebase quotas or billing limits before traffic reaches the configured thresholds.

Data model

Collection	Role
users	Identity, role, approval state, contact and candidate profile data
jobs	Public openings owned by approved recruiters
applications	Candidate submissions, links, cover notes, and status
reminders	Recipient-scoped recruiter and candidate notifications

Project structure

- `src/app/`: routes and page-level user interfaces.
- `src/components/`: shared authentication, branding, and reminder components.
- `src/lib/firebase-client.ts`: Firebase initialization and client access.
- `src/lib/jobnest-data.ts`: domain models, validation, persistence, and realtime state.
- `firestore.rules` and `firestore.indexes.json`: database security and query support.

Important operational notes

Never commit `.env.local`, OAuth secrets, service-account keys, or API tokens. External links must have suitable sharing permissions; JobNest does not control their privacy or availability. Real email/SMS delivery needs a separate provider. Firebase Storage is optional.